SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO4: Formulate data-driven web solutions using XML, XSLT, and JSP within the MVC framework. (L4)

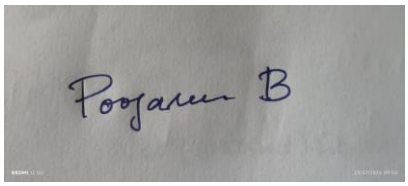
Assessment Tool: Pseudocode Writing Task (Algorithm Design)

Weightage: 20%

QUESTIONS: **Total Marks: 25**

Code of Conduct:

I POOJASREE B (192411091) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. XML Parsing Algorithm

Write a detailed pseudocode algorithm to parse an XML document that contains structured data such as user or product details. The algorithm should clearly describe the steps involved in loading the XML file, accessing its elements, traversing through nodes, extracting required

data, and displaying the output in a readable format. Ensure that the logic handles hierarchical structure properly.

```
ALGORITHM ParseXMLDocument


START

1. Specify the path of the XML file.

2. Check whether the XML file exists.
  IF file does not exist THEN
      Display "XML file not found"
      STOP
  END IF

3. Load the XML document.

4. Parse the XML document.
  IF parsing fails THEN
      Display "Invalid XML document"
      STOP
  END IF

5. Access the root element.

6. Traverse each child node of the root element.

7. FOR each child node DO

      IF the node contains child elements THEN
          Traverse the child elements recursively.

          FOR each nested element DO
              Identify the element name.
              Extract its value or attributes.
          END FOR

      ELSE
          Identify the element name.
          Extract its value.
      END IF

  END FOR



8. Extract the required information such as:
      ID, Name, Email, Price, Category, Quantity.

9. Format the extracted information.
```

```
10. Display the information in a readable format.

11. Release XML-related resources.

END
```

2. MVC Workflow

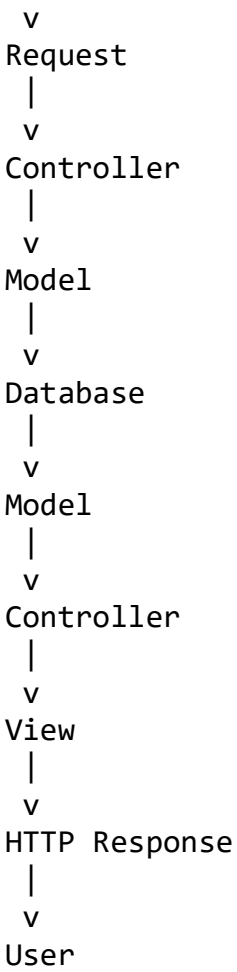
Write a pseudocode representation of the MVC (Model–View–Controller) workflow in a web application. The representation should clearly illustrate how a user request is received, processed by the Controller, how the Model interacts with the database, and how the View

generates and returns the response to the user. Include all major steps involved in request handling and response generation.

```
ALGORITHM MVCWorkflow

START

1. User sends a request through a web browser.

2. Web server receives the HTTP request.

3. Identify the requested URL and HTTP method.

4. Forward the request to the appropriate Controller.

5. Controller receives and validates the request.

6. Controller calls the appropriate Model or Service.

7. Model receives the request.

8. Model establishes a connection with the database.

9. Model executes the required database operation.

10. Database processes the query.

11. Database returns the requested data.

12. Model processes the retrieved data.

13. Model sends the data back to the Controller.

14. Controller selects the appropriate View.

15. Controller passes the data to the View.

16. View generates the dynamic HTML response.

17. View returns the generated response.

18. Web server sends the HTTP response to the browser.

19. Browser displays the response to the user.

END
```

MVC Flow

```
User
 |
```



3. JSP Request Flow

Write a detailed pseudocode algorithm to explain the request–response flow in a JSP-based web application. The algorithm should include steps such as client request initiation, server

processing, interaction with backend components such as Servlets or databases, and generation of dynamic content that is sent back to the client.

```
ALGORITHM JSPRequestFlow


START

1. User opens a web browser.

2. User enters the URL of a JSP page.

3. Browser creates an HTTP request.

4. Send the request to the web/application server.

5. Server receives the request.

6. Check whether the JSP page exists.

  IF JSP page does not exist THEN
      Return HTTP 404 error.
      STOP
  END IF

7. Check whether the JSP has already been translated and compiled.

  IF JSP is new or has been modified THEN
      Translate JSP into a Servlet.
      Compile the generated Servlet.
  END IF

8. Execute the generated Servlet.

9. Process the client request.

10. Communicate with backend components when required:
        - Servlets
        - JavaBeans
        - Business/service classes
        - Database

11. IF database access is required THEN
        Establish database connection.
        Execute SQL query.
        Retrieve required data.
        Release database resources.

   END IF

12. Return the retrieved data to the JSP/application layer.
```

13. JSP combines the data with HTML.

14. Generate the dynamic web page.

15. Create the HTTP response.

16. Send the response to the client's browser.

17. Browser receives the response.

18. Browser renders and displays the web page.

END

4. Database Retrieval

Write a pseudocode algorithm to retrieve data from a database. The algorithm should include steps for establishing a database connection, executing SQL queries, processing the result set,

and displaying the retrieved data. Clearly represent the sequence of operations involved in database interaction.

```
ALGORITHM RetrieveDataFromDatabase


START

1. Define database details:
      Database URL
      Username
      Password

2. Load the required database driver.

3. Establish a connection with the database.

4. IF connection fails THEN
      Display "Database connection failed"
      STOP
  END IF

5. Prepare the SQL query.

  Example:
      SELECT id, name, email FROM Users;

6. Create a Statement or PreparedStatement.

7. Execute the SQL query.

8. Store the returned records in a Result Set.

9. Check whether the Result Set contains records.

10. IF no records exist THEN
       Display "No records found"
   ELSE
       FOR each record in the Result Set DO
           Read ID.
           Read Name.
           Read Email.

           Display the retrieved information.
       END FOR
   END IF


11. Close the Result Set.

12. Close the Statement.
```

13. Close the database connection.

14. Display "Data retrieval completed".

END

5. PHP Validation

Write a pseudocode algorithm for validating user input in a web form using PHP concepts. The algorithm should include checks for empty fields, validation of input formats such as

email and password, handling invalid inputs with appropriate error messages, and allowing submission only when all validations are satisfied.

```
ALGORITHM ValidateUserInput


START

1. Initialize an empty error list.

2. Receive the form data submitted by the user.

3. Retrieve:
      Name
      Email
      Password
      ConfirmPassword

4. Remove unnecessary whitespace from the input.

5. Check whether Name is empty.

  IF Name is empty THEN
      Add "Name is required" to the error list.
  END IF

6. Check whether Email is empty.

  IF Email is empty THEN
      Add "Email is required" to the error list.
  ELSE
      Validate the email format.

      IF email format is invalid THEN
          Add "Enter a valid email address" to the error list.
      END IF
  END IF

7. Check whether Password is empty.

  IF Password is empty THEN
      Add "Password is required" to the error list.
  ELSE
      Check the password length.

      IF password length is less than the required minimum THEN
          Add "Password is too short" to the error list.

      END IF
  END IF

8. Check whether ConfirmPassword is empty.
```

```
  IF ConfirmPassword is empty THEN
      Add "Please confirm your password" to the error list.
  ELSE IF Password != ConfirmPassword THEN
      Add "Passwords do not match" to the error list.
  END IF

9. Check the error list.

  IF error list is not empty THEN
      Display the appropriate error messages.
      Return the user to the form.
      STOP
  END IF

10. If there are no errors THEN
       Process the valid input.
       Securely hash the password.
       Store the information in the database.

11. Display "Registration successful".

END
```